

DATA ANALYTICS

FASE 3 | AULA 05 -

TIPOS DE BANCOS NOSQL: DOCUMENTOS, CHAVE-VALOR, GRAFOS E COLUNAS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
MERCADO, CASES E TENDÊNCIAS	25
O QUE VOCÊ VIU NESTA AULA?	28
REFERÊNCIAS	29

EMANIP

O QUE VEM POR AÍ?

Nesta aula, exploraremos os principais tipos de bancos de dados NoSQL, que oferecem soluções alternativas aos bancos relacionais tradicionais. Focaremos nos modelos de **documentos**, **chave-valor**, **grafos** e **colunas**, compreendendo suas estruturas, casos de uso e vantagens. Ao final, você estará apto(a) a identificar qual modelo é mais adequado para diferentes cenários de aplicação.



HANDS ON

Nesta aula, exploraremos as quatro principais categorias de bancos de dados NoSQL – **Documentos, Chave-Valor, Grafos e Colunas** – com demonstrações práticas de suas aplicações no mercado. Na próxima seção, teremos um passo a passo para a instalação do MongoDB.



SAIBA MAIS

Passo a Passo para a instalação do MongoDB

Pré-requisitos:

- **Sistema Operacional:** certifique-se de que seu sistema é compatível. O MongoDB suporta Windows, macOS e Linux.
- **Ferramenta de linha de comando:** terminal, PowerShell ou CMD.
- **Pacote de instalação:** utilize o gerenciador de pacotes ou o site oficial.

Documentos (MongoDB):

- Instale o [MongoDB](#).
- Execute o MongoDB localmente usando o comando: `mongod --dbpath /path/to/your/db`.
- Use o **MongoDB Compass** (gratuito) para interação visual com o banco.

Instalação no Windows

Baixando o Instalador:

- Acesse o [site oficial](#).
- Escolha o instalador MSI para Windows.

Executar o Instalador:

- Clique no arquivo .msi baixado.
- Selecione a opção "Complete" durante a instalação.

Configurar o Serviço MongoDB: marque a opção "Install MongoDB as a Service" para que o MongoDB inicie automaticamente.

Adicionando Ferramentas de Linha de Comando: durante a instalação, marque a opção para instalar o **MongoDB CLI** (ou "MongoDB Shell").

Configuração de variáveis de ambiente:

- Adicione o diretório bin do MongoDB ao PATH.
- **Exemplo:** C:\Program Files\MongoDB\Server\<versão>\bin.

Iniciar o Servidor:

Abra o terminal e inicie o MongoDB com o comando “net start MongoDB”.

Conectar-se ao MongoDB:

- Inicie o shell MongoDB: “Mongosh”.

Instalação no macOS**Instalando o Homebrew:**

Caso não tenha o Homebrew instalado, instale-o utilizando o comando

```
Homebrew:/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Instalando o MongoDB

Utilize o Homebrew para instalar o MongoDB:

- brew tap mongodb/brew.
- brew install mongodb-community@6.0.

Inicie o serviço MongoDB: brew services start mongodb/brew/mongodb-community.

Acesse o Shell MongoDB: Mongosh

Instalação no Linux (Ubuntu/Debian)

- Importe a Chave Pública do MongoDB.
- **Adicione a chave GPG:** curl -fsSL https://pgp.mongodb.com/server-6.0.asc | sudo gpg --dearmor -o /usr/share/keyrings/mongodb-archive-keyring.gpg

Adicionando o repositório:

Inclua o repositório MongoDB ao seu sistema: echo "deb [signed-by=/usr/share/keyrings/mongodb-archive-keyring.gpg] https://repo.mongodb.org/apt/ubuntu focal/mongodb-org/6.0 multiverse" | sudo tee /etc/apt/sources.list.d/mongodb-org-6.0.list

- **Atualize a lista de pacotes:** sudo apt update.

- **Instale o servidor MongoDB:** `sudo apt install -y mongodb-org`.

Iniciar o MongoDB:

- **Inicie o serviço MongoDB:** `sudo systemctl start mongod`.

Habilitando o MongoDB para inicialização automática:

- **Configure o serviço para iniciar automaticamente:**
`sudo systemctl enable mongod`.
- **Acesse o MongoDB Shell:** `Mongosh`

Testando a Instalação

Após instalar e iniciar o MongoDB, você pode verificar se está tudo funcionando corretamente:

- **Abra o shell MongoDB:** `mongosh`.
- **Insira o comando para listar bancos de dados:** `"show dbs"`.

Dicas e Recursos

Ferramenta GUI: para gerenciar o MongoDB visualmente, instale o [MongoDB Compass](#).

Documentação Oficial: consulte a [Documentação MongoDB](#) para mais detalhes.

Exercício Prático

Documentos (MongoDB): insira um documento JSON.

```
db.customers.insertMany([
  { name: "João Silva", age: 30, purchases: ["Notebook", "Mouse"] },
  { name: "Maria Santos", age: 25, purchases: ["Smartphone"] },
  { name: "Carlos Almeida", age: 35, purchases: ["Tablet",
"Headphones"] }
])
```

Consulta com \$match e \$group:

Retorna a média de idade dos clientes:

```
db.customers.aggregate([  
  { $match: { age: { $gte: 18 } } },  
  { $group: { _id: null, averageAge: { $avg: "$age" } } }  
])
```

Passo a passo para instalação do Redis Open Source**Pré-requisitos:**

- **Sistema Operacional:** o Redis é compatível com Linux, macOS e Windows (com adaptações).
- **Ferramenta de linha de comando:** terminal, PowerShell ou CMD.
- **Pacote de instalação:** baixe do site oficial ou use gerenciadores de pacotes disponíveis para seu sistema.

Instalação no Linux (Ubuntu/Debian)

Para atualizar os repositórios: `sudo apt update`

Para instalar o Redis: `sudo apt install redis-server -y`

Editando as configurações (Opcional):

- **Abra o arquivo de configuração para ajuste de parâmetros (como o persistência):** `sudo nano /etc/redis/redis.conf`
- **Certifique-se de que o Redis está configurado para iniciar automaticamente:** `supervised systemd`.

Reiniciando o Serviço: `sudo systemctl restart redis`

Iniciando o Redis:

- **Para verificar se o Redis está funcionando:** `redis-cli ping`
- **Retorno esperado:** PONG
- **Habilitar o Redis na Inicialização:** `sudo systemctl enable redis`.

Instalação no Windows

O Redis não possui suporte nativo no Windows, mas há implementações compatíveis como o Memurai ou o WSL (Windows Subsystem for Linux).

Opção 1 - Usando WSL

- **Instalar o WSL:** no PowerShell, habilite o WSL: wsl-install.
- **Instalar o Redis no WSL:** siga o mesmo procedimento para Linux listado acima.

Opção 2 - Redis Nativo no Windows

- **Baixe o Redis para Windows:** use a versão compatível de [Memurai](#).
- **Executar o Servidor Redis:** após a instalação, inicie o servidor Redis no CMD ou Power Shell: redis-server.exe.

Conectar-se ao Redis:

- **No mesmo diretório, abra o cliente Redis:** redis-cli.exe.
- **Teste com o comando:** PING

Instalação no macOS

- **Instalar o Homebrew (se necessário):** caso não tenha o Homebrew instalado, instale-o utilizando o comando "Homebrew:/bin/bash -c "\$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)".
- **Instalar o Redis:** brew install redis.
- **Iniciar o Redis:** brew services start redis.

Testar a Conexão

- **Execute o cliente Redis:** redis-cli ping.
- **Retorno esperado:** PONG.

Testando o Redis após a instalação

Verifique se o Redis está em execução:

- No Linux ou macOS: ps aux | grep redis.
- No Windows (PowerShell): Get-Process redis-server.

Teste Operações Básicas: acesse o cliente Redis:redis-cli.

Execute comandos básicos:

- SET key "value".
- GET key.

Dicas Adicionais

Painel Web: para gerenciar o Redis visualmente, use ferramentas como Redis Insight.

Persistência: configure persistência ajustando os parâmetros “save” e append only” no arquivo de configuração (redis.conf).

Exercícios práticos

Criação de pares chave-valor:

```
SET user:1001 "João Silva"
```

```
SET user:1002 "Maria Santos"
```

Operações GET e SET:

```
GET user:1001
```

```
GET user:1002
```

Incremento de contadores (útil para tracking):

```
INCR page_views
```

Passo a passo para a instalação ou uso do Neo4j (Bancos de Grafos)

O Neo4j é uma das ferramentas de banco de grafos mais populares. Você pode usá-lo instalando localmente, executando via Docker ou usando o serviço on-line Neo4j AuraDB (sem instalação).

Opção 1 - Neo4j AuraDB (Uso On-line - sem instalação)

Crie uma conta no [Neo4j AuraDB](#):

- **Acesse o site:** Neo4j AuraDB.

- Crie uma conta gratuita ou faça login com um serviço como Google.

Criando um banco de dados:

- No painel principal, clique em “Create Free Database”.
- Escolha um nome, localidade e crie o banco de dados.

Conectando-se ao banco de dados:

- Você receberá um link para o banco junto com um nome de usuário (neo4j) e uma senha.
- Use a interface gráfica do Neo4j ou conecte-se via linha de comando.

Interagindo com o Banco

Use o Cypher Query Language para consultas diretamente no painel:

```
CREATE (a:Person {name: "Alice"})-[:FRIENDS_WITH]->(b:Person {name: "Bob"});  
  
MATCH (n) RETURN n;
```

Opção 2 - Instalação do Neo4j Localmente

Para Linux:

Adicionar o Repositório:

```
wget -O - https://debian.neo4j.com/neotechnology.gpg.key | sudo apt-key add -  
  
echo'deb https://debian.neo4j.com stable 4.x' | sudo tee  
/etc/apt/sources.list.d/neo4j.list
```

Atualizar e Instalar:

```
sudo apt update  
sudo apt install neo4j
```

Iniciar o Servidor:

```
sudo systemctl start neo4j
```

Configurar Senha Inicial:

Acesse o cliente via terminal:

cypher-shell

Use o login inicial (neo4j/neo4j) e defina uma nova senha.

Acessar o Painel Web:

- Abra no navegador: <http://localhost:7474>.
- Faça login com o nome de usuário e senha configurados.

Para Windows

Baixe o Instalador:

- Faça o download do Neo4j Desktop: [Neo4j Download](#).

Instalar o Neo4j:

- Siga o assistente de instalação.
- Escolha o diretório para o banco de dados.

Iniciar o Banco:

- Abra o Neo4j Desktop.
- Crie um projeto e inicie um banco de dados.

Acessar o Banco:

Use o painel do Neo4j Desktop ou o navegador em <http://localhost:7474>.

Para macOS

Instalar via Homebrew:

Caso não tenha o Homebrew instalado, instale-o primeiro utilizando o comando `:/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"`

- **Instalar o Neo4j:** `brewinstall neo4j`.
- **Iniciar o Serviço:** `neo4j start`.
- **Acessar o Painel Web:** abra no navegador: <http://localhost:7474>.

Opção 3 - Neo4j via Docker

Instalar o Docker:

Siga as instruções para instalar o Docker: Docker Installation.

Executar o Neo4j no Docker:

```
docker run \
  --name neo4j \
  -p 7474:7474 -p 7687:7687 \
  -d neo4j
```

Acessar o Painel Web:

- Abra o navegador e vá para <http://localhost:7474>.
- Login padrão: neo4j/neo4j (você será solicitado a alterar a senha).

Interação com Neo4j (Comandos Básicos em Cypher)

Criar Nós e Relações: CREATE (a:Person {name: "Alice"})-[:KNOWS]->(b:Person {name: "Bob"});

Consultar Nós: MATCH (n:Person) RETURN n.name;

Excluir Todos os Nós: MATCH (n) DETACH DELETE n;

Contar Relações: MATCH (a)-[r]->(b) RETURN COUNT(r);

Dicas Adicionais

- Use o Neo4j Browser para executar consultas e visualizar grafos.
- Ferramentas como Bloom oferecem visualização mais avançada.
- Leia a documentação oficial para explorar recursos mais complexos.

Passo a passo para instalação ou uso do Cassandra (Apache Cassandra)

Você pode usar o Apache Cassandra localmente, via Docker ou na nuvem por meio de plataformas gratuitas como o **AstraDB**, uma solução de Cassandra oferecida pela DataStax.

Opção 1: uso do Cassandra na nuvem com AstraDB (Sem Instalação)

Passo 1: Criar Conta no AstraDB.

- Acesse o site oficial: AstraDB.
- Clique em “**Sign Up**” e crie uma conta gratuita usando seu e-mail ou login com Google/GitHub.

Passo 2: Criar um banco de dados Cassandra.

No painel, clique em “**Create Database**” e escolha:

- **Provider:** AWS, GCP ou Azure.
- **Region:** Escolha uma região próxima para baixa latência.

Nomeie o banco de dados e clique em “**Create Database**”.

Passo 3: Acessar o Banco.

1. Quando o banco estiver ativo (status “Running”), clique nele para visualizar detalhes.
2. Use o **CQL Console** diretamente pelo painel ou baixe as credenciais para conexão externa.

Passo 4: Conectar-se ao Cassandra.

1. Clique em “**Connect**” e selecione “**CQL Console**” ou baixe o bundle de configuração para uso com ferramentas locais.
2. Execute comandos no console CQL para interagir com o banco:

```
sql

CREATETABLEsensor_data (
    id UUID PRIMARY KEY,
    timestampTIMESTAMP,
    temperature FLOAT
);

INSERTINTOsensor_data (id, timestamp, temperature)
VALUES (uuid(), toTimestamp(now()), 23.5);
```

```
SELECT*FROMsensor_data;
```

Opção 2: Instalação local do cassandra

Para Linux:

Adicione o repositório oficial:

```
bash
```

```
echo"deb https://www.apache.org/dyn/closer.lua/cassandra/debian 311x main"  
| sudo tee -a /etc/apt/sources.list.d/cassandra.sources.list  
curl https://downloads.apache.org/cassandra/KEYS | sudo apt-key add -
```

Instale o Cassandra:

```
bash
```

```
sudo apt update  
sudo apt install cassandra
```

Inicie o serviço:

```
bash
```

```
sudo systemctl start cassandra  
sudo systemctl enable cassandra
```

Teste a conexão. Para isso use o cliente cqlsh:

```
bash
```

```
cqlsh
```

Execute comandos simples:

```
sql
```

```
SHOW VERSION;
```

Para macOS

Instale via Homebrew:

```
bash
```

```
brew install cassandra
```

Inicie o Cassandra:

```
bash
```

```
cassandra -f
```

Teste com cqlsh:

```
bash
```

```
cqlsh
```

Para Windows

Instale o gerenciador de pacotes Chocolatey caso ainda não possua:

```
powershell
```

```
Set-ExecutionPolicy Bypass -Scope Process -Force
```

```
[System.Net.ServicePointManager]::SecurityProtocol
```

```
=
```

```
[System.Net.ServicePointManager]::SecurityProtocol -bor 3072
```

```
iex
```

```
((New-Object
```

```
System.Net.WebClient).DownloadString('https://chocolatey.org/install.ps1'))
```

Instale o Cassandra:


```
powershell
```

```
choco installcassandra
```

Inicie o Cassandra: navegue até o diretório de instalação e execute:

```
cmd
```

```
cassandra.bat
```

Use o cqlsh para conexão:

```
cmd
```

```
cqlsh
```

Opção 3 – Use o Cassandra com Docker

Baixe e instale o Docker: Docker Installation.

Execute o Cassandra:

```
bash
```

Copiar código

```
dockerrun --namecassandra -p 9042:9042 -d cassandra
```

Conectar-se ao Cassandra e acesse o container:

```
bash
```

Copiar código

```
dockerexec -it cassandrabash
```

Execute o cliente cqlsh:

```
bash
```

Copiar código

```
cqlsh
```

Teste o banco, crie uma tabela e insira dados:

```
sql

Copiar código

CREATETABLE users (id UUID PRIMARY KEY, name TEXT);

INSERTINTO users (id, name) VALUES (uuid(), 'Alice');

SELECT*FROMUsers;
```

Dicas para todos os métodos

Ferramentas GUI: use o DataStax Studio para interagir com o Cassandra visualmente.

Documentação Oficial: consulte a [Documentação Apache Cassandra](#) para detalhes avançados.

Exercícios práticos

Grafos (Neo4j): modelo de rede social.

```
cypher

CREATE (joao:Person {name: "João"}),
      (maria:Person {name: "Maria"}),
      (carlos:Person {name: "Carlos"}),
      (joao)-[:FRIENDS_WITH]->(maria),
      (maria)-[:FRIENDS_WITH]->(carlos)
```

Consulta para encontrar conexões de João:

```
cypher

MATCH (joao:Person {name: "João"})-[:FRIENDS_WITH]->(friends)

RETURN friends.name
```

Exercícios Práticos

Colunas (Cassandra): criação de tabela para IoT.

```
sql

CREATE TABLE sensor_data (
  sensor_id UUID PRIMARY KEY,
  timestamp TIMESTAMP,
  temperature FLOAT,
  humidity FLOAT
);
```

Inserção de dados:

```
sql

INSERT INTO sensor_data (sensor_id, timestamp, temperature, humidity)
VALUES (uuid(), toTimestamp(now()), 22.5, 60.0);
```

Consulta de dados mais recentes:

```
sql

SELECT * FROM sensor_data WHERE sensor_id = <UUID> ORDER BY
timestamp DESC LIMIT 5;
```

Entendendo os Tipos de Bancos NoSQL

O NoSQL engloba quatro principais categorias de bancos de dados: **documentos**, **chave-valor**, **grafos** e **colunas**. Cada uma é otimizada para casos de uso específicos, substituindo a rigidez dos bancos relacionais (SQL) por flexibilidade e desempenho.

Bancos de Documentos

Estrutura: armazenam dados em formato JSON, BSON ou XML. Cada documento contém pares chave-valor e pode ser estruturado hierarquicamente.

Características:

- Schema flexível, ideal para dados semi-estruturados.
- Rápido para leitura e escrita.
- Suporta consultas complexas.

Exemplo de Uso: MongoDB.

Casos de Uso:

- Aplicações web dinâmicas.
- Catálogos de produtos.
- Gerenciamento de conteúdo (CMS).

Exemplo Visual: estrutura de documento

```
{  
  "userId": 1,  
  "name": "João Silva",  
  "purchases": [  
    { "item": "Notebook", "price": 3000 },  
    { "item": "Mouse", "price": 100 }  
  ]  
}
```

Bancos de Chave-Valor

Estrutura: armazenam pares simples de chave e valor, como um grande dicionário.

Características:

- Altíssima performance para leituras e gravações.
- Simples e eficiente, mas limitado para consultas complexas.
- Ideal para cache e dados de sessão.

Exemplo de Uso: Redis.

Casos de Uso:

- Sistemas de cache (por exemplo, aceleração de sites).
- Gerenciamento de sessões de usuário.
- Filas de mensagens e tracking de eventos.

Fluxo Operacional:

Chave → Valor

arduino

"user:123" → "João Silva"

"page_views" → "15"

Vantagens e Limitações:

Vantagem	Limitação
Rápido e simples	Não adequado para consultas complexas
Fácil de escalar horizontalmente	Não suporta joins ou relacionamentos

Bancos de Grafos

Estrutura: dados armazenados como nós (entidades) e arestas (relações).

Características:

- Otimizado para consultas sobre relações complexas.
- Linguagem específica, como a Cypher, para consultas.
- Escalabilidade limitada em grandes volumes de dados altamente conectados.

Exemplo de Uso: Neo4j.

Casos de Uso:

- Redes sociais (amizades, seguidores).
- Recomendação de conteúdo (sistemas de recomendação).
- Análise de fraudes (relacionamento entre transações).

Fluxo Operacional:**Exemplo: rede social**

CSS

[João]---[:FRIEND]---[Maria]---[:FOLLOW]---[Carlos]

Vantagens e limitações

Vantagem	Limitação
Excelente para dados relacionais.	Desempenho pode ser limitado em consultas massivas.
Linguagem visual e intuitiva.	Escalabilidade mais complexa.

Bancos de Colunas

Estrutura: armazenam dados em um formato orientado a colunas, otimizando a leitura de colunas específicas em vez de linhas inteiras.

Características:

- Boa performance para grandes volumes de dados.
- Ideal para análises e consultas agregadas.
- Schema flexível, mas não tão dinâmico quanto documentos.
- **Exemplo de Uso:** Cassandra.

Casos de Uso:

- Aplicações IoT (dados de sensores).
- Registros de eventos e logs.
- Analytics de grandes volumes.

Estrutura Visual

Sensor ID	Timestamp	Temperatura	Umididade
001	2024-12-01 10:00	22.5°C	55%
002	2024-12-01 10:05	23.0°C	60%

Vantagens e Limitações

Vantagem	Limitação
Escalabilidade horizontal fácil.	Menos intuitivo para iniciantes.
Excelente para leitura de big data.	Não ideal para consultas relacionais.

Comparações

Tabela Comparativa de Características

Tipo	Flexibilidade	Desempenho	Melhor Uso
Documentos	Alta	Rápido	Aplicações dinâmicas.
Chave-Valor	Baixa	Altíssimo	Cache, sessões.
Grafos	Média	Boa para relações	Redes sociais, recomendações.
Colunas	Média	Rápido para análise	Analytics, IoT.

Gráfico de Decisão

Necessidade Principal:

- Alta Flexibilidade → Bancos de Documentos.
- Velocidade Simples → Bancos de Chave-Valor.
- Relações Complexas → Bancos de Grafos.
- Big Data e Análise → Bancos de Colunas.

Quando Escolher Cada Tipo

Documentos:

- Use para dados não estruturados ou semi-estruturados que variam muito.
- Exemplos: e-commerce, blogs, gerenciamento de conteúdo.

Chave-Valor:

- Ideal para armazenamento de dados temporários e rápidos.
- Exemplos: cache, estados de sessão, contadores.

Grafos:

- Escolha para redes sociais, análises de grafos e recomendações.
- Exemplos: amizades no Facebook, conexões no LinkedIn.

Colunas:

- Perfeito para armazenar e processar grandes volumes de dados em larga escala.
- Exemplos: logs de servidores, medições IoT, análises empresariais.

EMANDA

MERCADO, CASES E TENDÊNCIAS

O mercado de bancos de dados NoSQL tem crescido exponencialmente, impulsionado pela explosão de dados não estruturados e pela demanda por soluções escaláveis e flexíveis. Abaixo, exploramos o cenário atual, casos de sucesso e tendências que moldam o futuro dessa tecnologia.

Mercado

Crescimento do Mercado: segundo [relatórios](#) de mercado, o mercado global de NoSQL está projetado para atingir **US\$ 35 bilhões até 2027**, com uma taxa de crescimento anual composta (CAGR) de aproximadamente **30%**.

O principal impulsionador é a demanda por soluções de big data, especialmente em setores como e-commerce, IoT, fintech e redes sociais.

Setores que mais adotam NoSQL

E-commerce: para catálogos dinâmicos e recomendações personalizadas.

Redes sociais: modelagem de relações e comportamento do usuário.

Fintech: para análise de risco e detecção de fraudes em tempo real.

IoT: gerenciamento de grandes volumes de dados gerados por dispositivos conectados.

Principais Players

MongoDB: líder em bancos de documentos.

Redis: amplamente utilizado para soluções de cache.

Neo4j: domina o mercado de bancos de grafos.

Cassandra: popular em soluções de grande escala e distribuídas.

Cases de Sucesso - MongoDB (Documentos)

Empresa: Uber.

Aplicação: gerenciamento de dados de viagens em tempo real, integrando informações de motoristas, passageiros e rotas.

Impacto: redução de latência e escalabilidade para lidar com milhões de transações por dia.

Redis (Chave-Valor)

Empresa: Twitter.

Aplicação: cache de timelines e sessões de usuários.

Impacto: melhor desempenho e disponibilidade, permitindo atualizações instantâneas para milhões de usuários.

Neo4j (Grafos)

Empresa: LinkedIn.

Aplicação: modelagem de conexões profissionais (quem você conhece).

Impacto: análise eficiente de relações complexas, suportando algoritmos de recomendação personalizados.

Cassandra (Colunas)

Empresa: Netflix.

Aplicação: armazenamento de dados de streams, incluindo histórico de visualizações e preferências de usuários.

Impacto: alta disponibilidade e escalabilidade para atender a mais de 200 milhões de assinantes.

Tendências

Integração com IA e Machine Learning: bancos NoSQL estão sendo cada vez mais integrados a pipelines de Machine Learning, permitindo o armazenamento e o processamento de dados em tempo real para algoritmos de recomendação, previsões e análises comportamentais.

Hibridização entre SQL e NoSQL: surgem bancos híbridos, como o Google Spanner, que combinam a escalabilidade dos bancos NoSQL com a consistência transacional do SQL. Isso atende a aplicações que exigem o melhor dos dois mundos.

Adoção de Serverless: soluções serverless estão tornando o NoSQL mais acessível e escalável, com provedores como AWS DynamoDB e Google Firestore oferecendo modelos de pagamento por uso.

Maior Foco em Segurança: com a crescente adoção em setores como saúde e finanças, os bancos NoSQL estão implementando criptografia em repouso, autenticação robusta e controles granulares de acesso.

Expansão em Ambientes IoT: com a proliferação de dispositivos conectados, bancos NoSQL como Cassandra e MongoDB estão sendo amplamente utilizados para armazenar e processar dados IoT em tempo real.

Desafios do Mercado

Falta de Padronização: ao contrário dos bancos relacionais, os bancos NoSQL não seguem padrões universais, o que pode dificultar migrações e treinamentos.

Complexidade de Gerenciamento: bancos distribuídos, como Cassandra, podem ser complexos de configurar e monitorar, exigindo habilidades especializadas.

Custos: apesar de escaláveis, os custos podem aumentar rapidamente em soluções baseadas na nuvem, dependendo do volume de dados.

Futuro do Mercado

Expansão Global: adoção crescente em mercados emergentes como Índia e Brasil.

Automação: ferramentas avançadas para gerenciamento automático de escalabilidade, replicação e otimização.

Edge Computing: bancos NoSQL estão se adaptando para suportar aplicações de borda, onde a latência deve ser minimizada.

Os bancos NoSQL continuam a redefinir a forma como armazenamos e acessamos dados, oferecendo flexibilidade, desempenho e escalabilidade em um mundo cada vez mais orientado a dados. Organizações que adotam essas tecnologias ganham vantagem competitiva, especialmente em cenários onde a inovação e a agilidade são cruciais.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos o fascinante universo dos bancos de dados **NoSQL**, destacando suas quatro principais categorias: **documentos**, **chave-valor**, **grafos** e **colunas**. Esses tipos oferecem flexibilidade e escalabilidade que superam os bancos relacionais tradicionais em muitos cenários, e cada um com características únicas e casos de uso específicos.



REFERÊNCIAS

APACHE CASSANDRA. **Cassandra Documentation**. [s.d]. Disponível em: <https://cassandra.apache.org/doc/latest/>. Acesso em: 26 mar. 2025.

MONGODB. **Documentação MongoDB**. [s.d]. Disponível em: <https://www.mongodb.com/docs/>. Acesso em: 26 mar. 2025.

NEO4J. **Neo4j documentation**. Disponível em: <https://neo4j.com/docs/>. Acesso em: 26 mar. 2025.

REDIS. **Docs**. [s.d]. Disponível em: <https://redis.io/docs/>. Acesso em: 26 mar. 2025.

PALAVRAS-CHAVE

NoSQL. Bancos de Dados Flexíveis. Banco de Dados Escaláveis.

Documentos. Chave-Valor. Grafos. Colunas. Big Data. IoT.

EXEMPLO

The background is a dark blue field filled with numerous small, light blue dots. Overlaid on this are several large, flowing, wavy lines in shades of teal, blue, and yellow. These lines create a sense of movement and depth. Scattered throughout the composition are various geometric shapes: a circle containing the number '7', a small circle, an 'X' mark, a small circle, and a hexagon.

POSTECH